

GloVe: Global Vectors for Word Representation

Outline

- The co-occurrence matrix
- From counts to ratios
- Deriving the objective function
- The weighting function
- Optimization
- Relationship to matrix factorization
- Worked example
- Connections to other methods

Part I: The Co-occurrence Matrix

Starting Point: Co-occurrence Counts

GloVe starts from a **word-word co-occurrence matrix** X

- X_{ij} = number of times word j appears in a context window around word i
- Symmetric window of fixed size (e.g., 10 words left and right)
- Optional **distance weighting**: closer words contribute more ($1/d$)

The Matrix X

$$X_{ij} = \sum_{t=1}^T \mathbf{1}[x_t = i] \sum_{\substack{l=-L \\ l \neq 0}}^L \frac{\mathbf{1}[x_{t+l} = j]}{|l|}$$

- X is **symmetric**: $X_{ij} = X_{ji}$
- Row sum $X_i = \sum_k X_{ik}$: total context count for word i

Co-occurrence Probabilities

Define the co-occurrence probability:

$$P_{ij} = P(j \mid i) = \frac{X_{ij}}{X_i}$$

- Probability that word j appears in the context of word i
- Same distributional signal as spectral embedding, but retains full counts

Connection to Spectral Embedding

Both methods start from **co-occurrence statistics**

- Spectral embedding: counts $\rightarrow$ Dice similarity $\rightarrow$ graph Laplacian $\rightarrow$ eigenvectors
- GloVe: counts $\rightarrow$ log transform $\rightarrow$ weighted least-squares factorization

GloVe works **directly** with the co-occurrence matrix

Part II: From Counts to Ratios

The Problem with Raw Probabilities

$P(k \mid \text{ice})$ tells us how often k appears near "ice"

But this conflates:

- The relationship between k and "ice"
- The overall frequency of k

Common words like "the" have high $P(\text{the} \mid w)$ for every w

The Key Insight: Ratios

The ratio $P(k \mid \text{ice})/P(k \mid \text{steam})$ **cancels out noise**

- k related to ice only $\rightarrow$ ratio $\gg 1$
- k related to steam only $\rightarrow$ ratio $\ll 1$
- k related to both $\rightarrow$ ratio ≈ 1
- k related to neither $\rightarrow$ ratio ≈ 1

The Ice/Steam Example

Probe k	$P(k \mid \text{ice})$	$P(k \mid \text{steam})$	Ratio
solid	1.9×10^{-4}	2.2×10^{-5}	8.9
gas	6.6×10^{-5}	7.8×10^{-4}	0.085
water	3.0×10^{-3}	2.2×10^{-3}	1.36
fashion	1.7×10^{-5}	1.8×10^{-5}	0.96

Four Cases from Ratios

Ratio $\gg 1$ (solid): probe related to "ice" but not "steam"

Ratio $\ll 1$ (gas): probe related to "steam" but not "ice"

Ratio ≈ 1 (water): probe related to both

Ratio ≈ 1 (fashion): probe related to neither

Ratios distinguish all four cases; raw probabilities cannot

Formalizing the Insight

Want a function of word vectors that captures ratios:

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

- w_i, w_j : **word vectors** for the two target words
- $\tilde{w}_k$: **context vector** for the probe word

Part III: Deriving the Objective

Step 1: Vector Differences

Ratios measure the *difference* between two words → encode via subtraction:

$$F(w_i - w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

Step 2: Dot Product

Need a scalar output $\rightarrow$ use the dot product:

$$F((w_i - w_j)^\top \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

Now $F : \mathbb{R} \rightarrow \mathbb{R}_{>0}$

Step 3: The Homomorphism Requirement

X is symmetric $\rightarrow$ word and context roles interchangeable

F must satisfy: $F(a + b) = F(a) \cdot F(b)$

The unique continuous solution: $F = \exp$

Step 4: Apply the Exponential

$$\exp(w_i^\top \tilde{w}_k) = P_{ik} = \frac{X_{ik}}{X_i}$$

Taking logarithms:

$$w_i^\top \tilde{w}_k = \log X_{ik} - \log X_i$$

Step 5: Absorb Normalization into Biases

$\log X_i$ depends only on word $i \rightarrow$ absorb into bias b_i

$$w_i^\top \tilde{w}_k + b_i + \tilde{b}_k = \log X_{ik}$$

The **GloVe model equation**: dot product + biases $\approx$ log count

Step 6: Weighted Least-Squares Objective

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

- $f(X_{ij})$: weighting function
- Ensures $X_{ij} = 0$ pairs do not contribute ($\log 0$ undefined)
- Prevents function words from dominating

Part IV: The Weighting Function

Design Requirements for f

Three conditions:

1. $f(0) = 0$ — unobserved pairs contribute nothing
2. **Non-decreasing** — more frequent co-occurrences carry more evidence
3. **Bounded** — extremely frequent pairs should not dominate

The GloVe Weighting Function

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$$

- $x_{\max} = 100$: saturation threshold
- $\alpha = 3/4$: sublinear exponent

Why $\alpha = 3/4$?

Linear weighting ($\alpha = 1$): high-frequency pairs still dominate

Sublinear ($\alpha = 3/4$): compresses the range

- Count 100 $\rightarrow$ weight 1.0
- Count 50 $\rightarrow$ weight 0.59 (not 0.50)

Moderate-frequency co-occurrences get **relatively more influence**

Why Weighting Matters

Without weighting, loss is dominated by: "the-of," "the-and," "in-the" ...

These carry **little semantic information**

The weighting function amplifies meaningful co-occurrences like "ice-solid" or "king-crown"

Part V: Optimization

Four Sets of Parameters

- **Word vectors** $w_i \in \mathbb{R}^d$ for each word
- **Context vectors** $\tilde{w}_j \in \mathbb{R}^d$ for each word
- **Word biases** $b_i \in \mathbb{R}$
- **Context biases** $\tilde{b}_j \in \mathbb{R}$

Total: $2|V|(d + 1)$ parameters

Gradients

For pair (i, j) with residual $\Delta_{ij} = w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij}$:

$$\frac{\partial J_{ij}}{\partial w_i} = 2 f(X_{ij}) \Delta_{ij} \tilde{w}_j$$

$$\frac{\partial J_{ij}}{\partial b_i} = 2 f(X_{ij}) \Delta_{ij}$$

Symmetric expressions for $\tilde{w}_j$ and $\tilde{b}_j$

AdaGrad

Adaptive learning rates for each parameter:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

- $G_t = \sum_{\tau=1}^t g_{\tau}^2$: accumulated squared gradients
- Common words $\rightarrow$ smaller effective learning rate
- Rare words $\rightarrow$ larger effective learning rate

A Batch Method

GloVe is fundamentally a **batch** method:

1. Build full co-occurrence matrix X from the corpus
2. Iterate over nonzero entries of X , computing loss and gradients
3. Converges in 50–100 passes

All contextual information is distilled into counts **before** optimization begins

Final Embedding

Two vectors per word: w_i (word) and $\tilde{w}_i$ (context)

Final embedding is their **sum**:

$$w_i^{\text{final}} = w_i + \tilde{w}_i$$

Averaging consistently improves performance over either set alone

Part VI: Matrix Factorization

The Core Factorization

Strip away biases:

$$W\tilde{W}^\top \approx \log X$$

- $W \in \mathbb{R}^{|V| \times d}$: word vector matrix
- $\tilde{W} \in \mathbb{R}^{|V| \times d}$: context vector matrix

This is **low-rank matrix factorization** of the log-count matrix

Comparison to SVD/LSA

	LSA	GloVe
Input	Raw counts (or TF-IDF)	Log counts
Weighting	Uniform	$f(X_{ij})$
Method	SVD (exact)	AdaGrad (approximate)
Solution	Globally optimal	Depends on initialization

Connection to PMI

$$\log X_{ij} = \text{PMI}(i, j) + \log X_i + \log X_j - \log |\text{corpus}|$$

Biases b_i and $\tilde{b}_j$ absorb the word-specific terms

Word vectors $w_i^\top \tilde{w}_j$ effectively approximate **pointwise mutual information**

GloVe and Word2Vec Converge

Levy and Goldberg (2014): Word2Vec skip-gram implicitly factorizes a shifted PMI matrix

Both GloVe and Word2Vec approximate the **same underlying quantity**

- GloVe: counts first, then optimizes
- Word2Vec: optimizes directly on the corpus

Part VII: Worked Example

The Toy Corpus

Same corpus as spectral embedding article:

1. "the cat sat on the mat"
2. "the dog sat on the rug"
3. "a cat chased a dog"
4. "the bird sat on the branch"
5. "a dog chased a bird"

Context window size = 1 (immediate neighbors)

Co-occurrence Matrix (Content Words)

	cat	dog	bird	sat	chased
cat	0	1	0	1	1
dog	1	0	1	1	1
bird	0	1	0	1	1
sat	1	1	1	0	0
chased	1	1	1	0	0

Probability Ratios

For "cat" vs "bird," all ratios equal 1.0:

$$\frac{P(\text{sat} \mid \text{cat})}{P(\text{sat} \mid \text{bird})} = \frac{1/3}{1/3} = 1.0$$

Identical distributional profiles → GloVe learns **similar vectors**

A GloVe Loss Term

For (cat, sat) with $X = 1$:

$$f(1) = (1/100)^{3/4} \approx 0.032$$

$$J = 0.032 \cdot (w_{\text{cat}}^\top \tilde{w}_{\text{sat}} + b_{\text{cat}} + \tilde{b}_{\text{sat}})^2$$

Since $\log 1 = 0$, model is driven to make the dot product + biases ≈ 0

Part VIII: Connections

The Count-Based Spectrum

LSA: raw counts $\rightarrow$ SVD

PMI-SVD: PMI-transformed counts $\rightarrow$ SVD

Spectral embedding: counts $\rightarrow$ similarity graph $\rightarrow$ Laplacian eigenvectors

GloVe: log counts $\rightarrow$ weighted least-squares factorization

Each refines the treatment of co-occurrence statistics

Forward: Prediction-Based Methods

Word2Vec (later article): trains a neural network to predict context words

- No explicit co-occurrence matrix
- Optimizes directly on the corpus
- Yet implicitly factorizes a shifted PMI matrix

Different computation, same underlying signal

The Count/Prediction Bridge

GloVe bridges count-based and prediction-based methods:

- **Count-based** in its data: starts from full co-occurrence matrix
- **Prediction-like** in its optimization: uses stochastic gradient descent

The distinction is computational, not statistical

Summary

- **Co-occurrence matrix** X_{ij} : counts with symmetric context window
- **Probability ratios** P_{ik}/P_{jk} discriminate word relationships
- **Model equation**: $w_i^\top \tilde{w}_k + b_i + \tilde{b}_k = \log X_{ik}$
- **Weighting** $f(x) = \min((x/x_{\max})^\alpha, 1)$ prevents function-word dominance
- **Low-rank factorization**: $W\tilde{W}^\top \approx \log X$
- **Biases absorb** word-frequency terms; vectors approximate PMI
- GloVe bridges **count-based** data with **prediction-like** optimization